

Formalization of Model Theory in Lean

Yunus Maranki

September 14, 2026

Bachelorarbeit Mathematik

Betreuer: Prof. Dr. Philipp Hieronymi

Zweitgutachter: Prof. Dr. Floris van Doorn

MATHEMATISCHES INSTITUT

MATHEMATISCH-NATURWISSENSCHAFTLICHE FAKULTÄT DER
RHEINISCHEN FRIEDRICH-WILHELMS-UNIVERSITÄT BONN

Zusammenfassung in deutscher Sprache

Diese Bachelorarbeit befasst sich mit der Formalisierung von O-Minimalität mit dem Beweisassistenten Lean und dessen mathematischer Bibliothek mathlib.

Insbesondere formalisieren wir den Monotoniesatz für o-minimale Strukturen und eine Version des Extremwertsatzes für definierbar vollständige Strukturen.

Zur Automatisierung von einfachen Definierbarkeitsbeweisen implementieren wir eine neue Lean-Taktik basierend auf der Taktik *Aesop* und mathlibs Resultaten über Definierbarkeit.

Außerdem erklären wir die Grundlagen von Lean und die Implementation von Modelltheorie in mathlib in Kapitel 1.

Der vollständige Lean code unserer Formalisierung ist auf GitHub verfügbar: <https://github.com/YunusM235/O-Minimality>

Contents

Introduction	3
1 Lean and mathlib	5
1.1 Foundations of Lean	5
1.2 Model theory in mathlib	8
2 Definability and o-minimality in Lean	15
2.1 O-minimality and definable completeness	15
2.2 Automating definability proofs	16
3 Definable completeness	20
3.1 Intermediate value theorem	20
3.2 Extreme value theorem	20
3.3 Formalization	24
4 Monotonicity theorem	26
4.1 Definable Ramsey theorem	26
4.2 Proof of the monotonicity theorem	27
4.3 Formalization	30
Conclusion	34
Bibliography	36

Introduction

In 1984 Grothendieck wrote [1]:

“... general topology was developed (during the thirties and forties) by analysts and in order to meet the needs of analysis, not for topology per se, i.e. the study of the topological properties of the various geometrical shapes.”

He proposed the notion of a *tame topology* which would exclude wild phenomena such as nowhere-differentiable functions or space filling curves.

In 1998 van den Dries, who had introduced the concept of o-minimality,¹ argued that [3]:

“o-minimal structures provide an excellent framework for developing tame topology, or topologie modérée, as outlined in Grothendieck’s prophetic ‘Esquisse d’un Programme’ of 1984.”

Since then, o-minimality has had many significant applications outside of model theory. One such application is the proof of the André-Oort conjecture by Pila, Shankar and Tsimerman in 2021 [4], using the so-called Pila-Zannier strategy [5]. In 2026 Tsimerman was awarded the Fields medal, in part for “the recasting of o-minimality as a fundamental method of arithmetic and complex algebraic geometry” [6].

Let $\mathcal{R} = (R, <, \dots)$ be a densely ordered structure without endpoints.

Definition. We say $\mathcal{R}$ is **o-minimal** if each definable set $X \subseteq R$ is a finite union of points and intervals.

One example of an o-minimal structure is $(\mathbb{R}, <, +, \cdot, \exp)$ [7].

For this thesis we formalize the *monotonicity theorem* for o-minimal structures with the proof assistant Lean. Additionally we formalize some results about *definable completeness*, which is the definable analogue of Dedekind completeness.

Definition. We say $\mathcal{R}$ is **definably complete** if every nonempty bounded definable set has an infimum and a supremum.

All expansions of $(\mathbb{R}, <)$ are definably complete since the underlying set $\mathbb{R}$ is already Dedekind complete. But in general a dense linear order might not be Dedekind complete. Note that definable completeness follows from o-minimality. Many definable analogues of standard results from analysis can be shown for definably complete structures. We formalize definable versions of the intermediate value theorem and the extreme value theorem for

¹The term *o-minimal* was first used by Pillay and Steinhorn [2].

definably complete structures. The proofs in sections 3 and 4 mostly follow the proofs from the *tame geometry* lecture notes by Philipp Hieronymi [8].

For the formalization we are using the proof assistant Lean 4 and its mathematical library mathlib. Lean was primarily developed by Leonardo de Moura and it can also be used as a functional programming language [9]. Mathlib is an open-source project with more than 750 contributors and it includes results from a broad range of areas of mathematics. Some notable formalization projects based on mathlib are the formalization of Carleson's theorem [10] and the ongoing formalization of Fermat's last theorem [11].

The Lean code of our formalization is available at:
<https://github.com/YunusM235/0-Minimality>

1 Lean and mathlib

1.1 Foundations of Lean

Lean is based on the type theory of the theorem prover Rocq. Specifically it is a variant of *calculus of constructions* with inductive types. For a detailed explanation of Lean see [12].

In set theory we are working with sets that contain elements. Similarly, in type theory we have types that have terms.

In Lean everything is a term of some type, including types themselves.

For every $n \in \mathbb{N}$ there is `Type n` which is a term of `Type (n+1)`. If x is a term of some type M , we write $x : M$. Lean also has a special type called `Prop` which is a term of `Type 0`. This is the type of propositions, i.e. statements that are true or wrong. This includes statements like $1 + 1 = 2$ or $1 + 1 = 3$. Such propositions are also types themselves and their terms are proofs of the statement. In Lean any two proof terms of some proposition are definitionally equal, this is called *proof irrelevance*.

Lean checks the correctness of a proof for some proposition ($p : \text{Prop}$) by verifying that the type of what we provided is exactly p . So verifying a proof becomes the same thing as type checking. This connection is known as the *Curry–Howard isomorphism* [13].

The types `Prop`, `Type 0`, `Type 1`, ... are called *universes* and they are the only types whose terms are again types. In Lean they are also called *sorts*. `Sort 0` is `Prop` and `Sort 1` is `Type 0` and so on.

Given types A and B there is a type $A \rightarrow B$ whose terms are functions from A to B . For example the function `fun (x : ℕ) ↦ x^2` has type $\mathbb{N} \rightarrow \mathbb{N}$.

Lean also has special function types called dependent function types. The following theorem from Lean is such a dependent function.

`theorem Nat.zero_le : (n : Nat) → 0 ≤ n := ...`

It sends $n \in \mathbb{N}$ to a proof of $0 \leq n$. So the type of `Nat.zero_le n` depends on n .

Next we will explain inductive types. Inductive types are built from a list of constructors, which are rules for constructing terms of the type we want to define. The type then consists of all terms that can be obtained from finitely many applications of these constructors.

One example of an inductive type are the natural numbers in Lean.

```
inductive Nat : Type 0 where
| zero : Nat
| succ : Nat → Nat
```

Here `Nat.zero` represents 0 and `Nat.succ Nat.zero` is 1 and `Nat.succ (Nat.succ Nat.zero)` is 2 and so on.

Other examples are logical connectives like $\vee$ or $\exists$:

```
inductive Or (a b : Prop) : Prop where
| inl : a → Or a b
| inr : b → Or a b

inductive Exists {α : Sort u} (p : α → Prop) : Prop where
| intro : (w : α) → (h : p w) → Exists p
```

Note that `Exists.intro` is a dependent function.

Lean also allows the following notation for constructor arguments.

```
inductive Exists {α : Sort u} (p : α → Prop) : Prop where
| intro (w : α) (h : p w) : Exists p
```

In general, arguments of constructors of some inductive type T can be function types constructed from the type T itself and previously defined types. But T is not allowed to be left of any arrow. So the following definition is not allowed in Lean.

```
inductive T where
| mk (x : T → ℕ) : T
```

Since constructors are injective, this would contradict Cantor's theorem.

Lean has four built-in constants `Quot`, `Quot.mk`, `Quot.ind` and `Quot.lift` for constructing quotient types. They are used to build the type of equivalence classes of some equivalence relation. The class for types with an equivalence relation is called `Setoid`.

```
class Setoid (α : Sort u) where
r : α → α → Prop
iseqv : Equivalence r
```

The type of equivalence classes of a setoid can be obtained from `Quotient`.

```
def Quotient {α : Sort u} (s : Setoid α) := ...
```

The quotient map is called `Quotient.mk`.

```
def mk {α : Sort u} (s : Setoid α) (a : α) : Quotient s := ...
```

Lean adds an axiom that shows that the images of equivalent terms under the quotient map are equal.

```
axiom Quot.sound : ∀ {α : Sort u} {r : α → α → Prop} {a b : α},
  r a b → Quot.mk r a = Quot.mk r b

theorem Quotient.sound {α : Sort u} {s : Setoid α} {a b : α} :
  a ≈ b → Quotient.mk s a = Quotient.mk s b := Quot.sound
```

With these quotient constructions Lean shows function extensionality.

```
theorem funext {α : Sort u} {β : α → Sort v}
  {f g : (x : α) → β x} (h : ∀ x, f x = g x) : f = g := ...
```

The next axiom Lean adds is propositional extensionality.

```
axiom propext {a b : Prop} : (a ↔ b) → a = b
```

Together with functional extensionality this is used to define the theory of sets.

```
def Set (α : Type u) := α → Prop

def mem (x : α) (a : Set α) := a x

theorem setext {a b : Set α} (h : ∀ x, x ∈ a ↔ x ∈ b) : a = b :=
  funext (fun x => propext (h x))
```

The last axiom Lean adds is the axiom of choice.

```
class inductive Nonempty (α : Sort u) : Prop where
  | intro (val : α) : Nonempty α

axiom Classical.choice {α : Sort u} : Nonempty α → α
```

Using `Classical.choice`, `propext` and `funext` Lean derives the law of excluded middle:

```
theorem em (p : Prop) : p ∨ ¬p := ...
```

This result is known as Diaconescu’s theorem [14]. Together with `propext` it also follows that all propositions (p : `Prop`) are equal to `True` or `False`.

More about Lean’s metatheory can be found in the master’s thesis by Mario Carneiro [15]. He shows that Lean’s type theory is consistent if and only if $\text{ZFC} + \{\text{there are } n \text{ inaccessible cardinals} \mid n \in \mathbb{N}\}$ is consistent.

1.2 Model theory in mathlib

In this section we will explain the implementation of model theory in `mathlib`, which was mainly developed by Aaron Anderson, Jesse Michael Han and Floris van Doorn. The basic definitions are written in a style inspired by the `Flypitch` project [16] of Han and van Doorn.

Languages

A language is defined as a structure with two fields:

```
structure Language where
  Functions : ℕ → Type u
  Relations : ℕ → Type v
```

Let `L` be such a language. Then `L.Functions n` is the type of n -ary function symbols and `L.Relations n` is the type of n -ary relation symbols.

For two languages (L : `Language`) and (L' : `Language`) the sum of them is defined using the disjoint unions `L.Functions n ⊕ L'.Functions n` and `L.Relations n ⊕ L'.Relations n`.

```
def Language.sum (L : Language) (L' : Language) : Language :=
  ⟨fun n => L.Functions n ⊕ L'.Functions n,
    fun n => L.Relations n ⊕ L'.Relations n⟩
```

Structures

An L -Structure on some type M is defined as a class that contains interpretations of the function and relation symbols.

```

class Structure where
  funMap : ∀ {n}, L.Functions n → (Fin n → M) → M
  RelMap : ∀ {n}, L.Relations n → (Fin n → M) → Prop

```

$\text{Fin } n$ is the type of natural numbers less than n . So the type $\text{Fin } n \rightarrow M$ represents M^n . $(\text{Fin } n \rightarrow M) \rightarrow M$ represents the functions $f: M^n \rightarrow M$.

$(\text{Fin } n \rightarrow M) \rightarrow \text{Prop}$ is the type that maps an element from M^n to a term of the type Prop , which is equal to True or False .

For instances $[\text{L}.\text{Structure } M]$ and $[\text{L}'.\text{Structure } M]$, `mathlib` defines an instance of the structure on M for the sum of the two languages:

```

instance sumStructure : (L1.sum L2).Structure M where
  funMap := Sum.elim funMap funMap
  RelMap := Sum.elim RelMap RelMap

```

Syntax

Now we will explain how the syntax of terms and formulas is implemented in `mathlib`.

```

inductive Term (α : Type u') : Type max u u'
| var : α → Term α
| func : ∀ {l : ℕ} (_f : L.Functions l)
  (_ts : Fin l → Term α), Term α

```

$\text{L.Term } \alpha$ is the inductive type of L -Terms with variables indexed by α .

To define formulas, `mathlib` first defines `BoundedFormula`:

```

inductive BoundedFormula : ℕ → Type max u v u'
| falsum {n} : BoundedFormula n
| equal {n} (t1 t2 : L.Term (α ⊕ (Fin n))) : BoundedFormula n
| rel {n l : ℕ} (R : L.Relations l)
  (ts : Fin l → L.Term (α ⊕ (Fin n))) : BoundedFormula n
| imp {n} (f1 f2 : BoundedFormula n) : BoundedFormula n
| all {n} (f : BoundedFormula (n + 1)) : BoundedFormula n

```

This is the type of formulas with free variables indexed by α and variables that can be used for quantification indexed by $\text{Fin } n$. The type $\alpha \oplus (\text{Fin } n)$ is the disjoint union of the types α and $\text{Fin } n$.

Given a formula ($f : \text{BoundedFormula } (\alpha \ n + 1)$), where the variables available for quantification are indexed by $\text{Fin } (\alpha \ n + 1)$, the formula `all f` stands for $\forall x_n f$.

Using `BoundedFormula` we can define formulas, sentences and theories:

```
abbrev Formula := L.BoundedFormula  $\alpha$  0

abbrev Sentence := L.Formula Empty

abbrev Theory := Set L.Sentence
```

Negation and existential quantification are defined as follows.

```
def not ( $\varphi : \text{L.BoundedFormula } \alpha \ n$ ) : L.BoundedFormula  $\alpha \ n :=$ 
   $\varphi$ .imp  $\perp$ 

def ex ( $\varphi : \text{L.BoundedFormula } \alpha \ (n + 1)$ ) : L.BoundedFormula  $\alpha \ n :=$ 
   $\varphi$ .not.all.not
```

The formula $\top$ and conjunction/disjunction are defined as instances of `Top`, `Min` and `Max`. This allows us to use the notation $\top$ and $\varphi \sqcap \psi$ for $\varphi \wedge \psi$ and $\varphi \sqcup \psi$ for $\varphi \vee \psi$.

```
instance : Top (L.BoundedFormula  $\alpha \ n$ ) :=
  ⟨BoundedFormula.not  $\perp$ ⟩

instance : Min (L.BoundedFormula  $\alpha \ n$ ) :=
  ⟨fun f g => (f.imp g.not).not⟩

instance : Max (L.BoundedFormula  $\alpha \ n$ ) :=
  ⟨fun f g => f.not.imp g⟩
```

Now biimplication can be defined like this:

```
def iff ( $\varphi \ \psi : \text{L.BoundedFormula } \alpha \ n$ ) :=
   $\varphi$ .imp  $\psi \sqcap \psi$ .imp  $\varphi$ 
```

Semantics

From now on assume that we have $(M : \text{Type}^*)$ and $[\text{L.Structure } M]$ in the context.

For an interpretation of variables ($v : \alpha \rightarrow M$), the function `Term.realize v` maps an L -Term with variables indexed by α to its interpretation using v .

```

def Term.realize (v :  $\alpha \rightarrow M$ ) : L.Term  $\alpha \rightarrow M$ 
| var k => v k
| func f ts => funMap f fun i => (ts i).realize v

```

Given $(\varphi : \text{L.BoundedFormula } \alpha \text{ l})$ and variable interpretations $(v : \alpha \rightarrow M)$ and $(xs : \text{Fin l} \rightarrow M)$ the proposition $\text{BoundedFormula.Realize } \varphi \text{ v xs}$ is **True** if $M \models \varphi(v, xs)$ and **False** otherwise.

```

def BoundedFormula.Realize :  $\forall \{l\}$  (_f : L.BoundedFormula  $\alpha$  l)
  (_v :  $\alpha \rightarrow M$ ) (_xs : Fin l  $\rightarrow M$ ), Prop
| _, falsum, _v, _xs => False
| _, equal t1 t2, v, xs => t1.realize
  (Sum.elim v xs) = t2.realize (Sum.elim v xs)
| _, rel R ts, v, xs => RelMap R fun i =>
  (ts i).realize (Sum.elim v xs)
| _, imp f1 f2, v, xs => Realize f1 v xs  $\rightarrow$  Realize f2 v xs
| _, all f, v, xs =>  $\forall x : M$ , Realize f v (snoc xs x)

```

This is used to define satisfaction for normal formulas, sentences and theories:

```

def Formula.Realize ( $\varphi : \text{L.Formula } \alpha$ ) (v :  $\alpha \rightarrow M$ ) : Prop :=
   $\varphi$ .Realize v default

def Sentence.Realize ( $\varphi : \text{L.Sentence}$ ) : Prop :=
   $\varphi$ .Realize (default :  $\_ \rightarrow M$ )

class Theory.Model (T : L.Theory) : Prop where
  realize_of_mem :  $\forall \varphi \in T$ ,  $M \models \varphi$ 

```

Note that $M \models \varphi$ here is the same as $\text{Sentence.Realize } M \varphi$.

Definability

Next we show how definability with parameters from a subset $A \subseteq M$ is implemented.

For this mathlib defines the structure on M where elements from A are added as a constant to the language.

Adding constant symbols indexed by some type $(\alpha : \text{Type*})$ to a language $(L : \text{Language})$ is defined as follows.

```

def constantsOnFunc ( $\alpha$  : Type*) :  $\mathbb{N} \rightarrow$  Type*
| 0 =>  $\alpha$ 
| ( $_ + 1$ ) => PEmpty

def constantsOn ( $\alpha$  : Type*) : Language :=
  ⟨constantsOnFunc  $\alpha$ , fun _ => Empty⟩

```

For $A \subseteq M$ mathlib now defines the structure on M consisting only of the elements from A as constants:

```

def constantsOn.structure (f :  $\alpha \rightarrow M$ ) :
  (constantsOn  $\alpha$ ).Structure M where
  funMap := fun {n} c _ =>
    match n, c with
    | 0, c => f c

instance paramsStructure (A : Set  $\alpha$ ) :
  (constantsOn A).Structure  $\alpha$  := constantsOn.structure (↑)

```

Assume that we have $[L.Structure\ M]$ and some subset $(A : Set\ M)$. Then we get the structure $[(L.sum\ (constantsOn\ A)).Structure\ M]$ where the elements of A are added as a constant:

```

example [L.Structure M] (A : Set M) :
  (L.sum (constantsOn A)).Structure M := inferInstance

```

Lean's type class inference delivers this structure since we have registered `paramsStructure` and `sumStructure`.

Using this mathlib now implements definability of sets :

```

def Definable (s : Set ( $\alpha \rightarrow M$ )) : Prop :=
   $\exists \varphi : (L.sum (constantsOn A)).Formula\ \alpha, s = \{v \mid \varphi.Realize\ v\}$ 

```

Mathlib has many basic results about definable sets, such as:

```

theorem Definable.inter {f g : Set (α → M)} (hf : A.Definable L f)
  (hg : A.Definable L g) :
  A.Definable L (f ∩ g) := by ...

theorem Definable.compl {s : Set (α → M)} (hf : A.Definable L s) :
  A.Definable L sc := by ...

lemma Definable.exists_of_finite [Finite β]
  {S : Set ((α ⊕ β) → M)} (hS : A.Definable L S) :
  A.Definable L
    { v : α → M | ∃ u : β → M, Sum.elim v u ∈ S } := by ...

```

Order

The language **order** consists of one binary relation representing $\leq$.

```

inductive orderRel : ℕ → Type
| le : orderRel 2

def order : Language := ⟨fun _ => Empty, orderRel⟩

```

An instance of the class **IsOrdered** for some language consists of the symbol representing $\leq$ in that language:

```

class IsOrdered (L : Language) where
  leSymb : L.Relations 2

```

This is then used for the definition of the formulas $t_1 \leq t_2$ and $t_1 < t_2$.

```

def Term.le [L.IsOrdered] (t1 t2 : L.Term (α ⊕ (Fin n))) :
  L.BoundedFormula α n := leSymb.boundedFormula2 t1 t2

def Term.lt [L.IsOrdered] (t1 t2 : L.Term (α ⊕ (Fin n))) :
  L.BoundedFormula α n := t1.le t2 ∧ ¬(t2.le t1)

```

Assume we have `[L.IsOrdered]`, `[L.Structure M]` and `[LE M]`, which means that the relation $\leq$ is defined on the type `M`. Then the class `OrderedStructure` says that this $\leq$ relation of `M` and the interpretation of the $\leq$ symbol from the language are the same.

```
class OrderedStructure [L.IsOrdered] [LE M] [L.Structure M] : Prop
  where
    relMap_leSymb : ∀ (x : Fin 2 → M), RelMap (leSymb : L.Relations
      2) x ↔ (x 0 ≤ x 1)
```

Mathlib also contains the theory of dense linear orders and results such as $\aleph_0$ -categoricity of DLO:

```
def dlo : L.Theory := L.linearOrderTheory ∪
  {L.noTopOrderSentence, L.noBotOrderSentence,
   L.denselyOrderedSentence}

theorem aleph0_categorical_dlo :
  (ℵ₀).Categorical Language.order.dlo := by ...
```

2 Definability and o-minimality in Lean

2.1 O-minimality and definable completeness

There is a formalization of o-minimality from 2020 by Reid Barton and Johan Commelin [17]. Their project is written in the older version Lean 3 instead of today's version Lean 4.

They used definability without logic, whereas we are working with mathlib4's model-theoretic `Set.Definable`. Their project includes some basic results about o-minimality and they show that semilinear sets form an o-minimal structure.

We will define o-minimality similar to how they define it.

First we define *basic* sets as subsets of M that are either a point or an open interval.

```
inductive IsBasic : Set M → Prop
| point (p : M) : IsBasic {p}
| Ioo (a b : M) (h : a < b) : IsBasic (Set.Ioo a b)
| Ioi (a : M) : IsBasic (Set.Ioi a)
| Iio (b : M) : IsBasic (Set.Iio b)
| Iii : IsBasic (Set.univ)
```

Now we define tame sets as subsets of M that are a finite union of basic sets.

```
def IsTame (s : Set M) : Prop :=
  ∃ (A : Finset (Set M)), (∀ a ∈ A, IsBasic a) ∧ s = ⋃ a ∈ A, a
```

And finally, a structure is o-minimal if all its definable sets $X \subseteq M$ are tame.

```
class OMinimal (L : Language) (M : Type*) [L.IsOrdered]
  [L.Structure M] [M ⊨ L.dlo]
  [LinearOrder M] [L.OrderedStructure M] : Prop where
  is_omimal : ∀ {s : Set M}, Set.univ.Definable₁ L s → IsTame s
```

The definable completeness class consists of one field for the supremum:

```
class DefinablyComplete (L : Language) (M : Type*) [L.IsOrdered]
  [L.Structure M] [M ⊨ L.dlo]
  [LinearOrder M] [L.OrderedStructure M] : Prop where
  lub : ∀ (s : Set M), s.Nonempty → Set.univ.Definable₁ L s
    → BddAbove s → ∃ b, IsLUB s b
```

This definition doesn't include the infimum since it can be obtained by taking the supremum of the lower bounds:

```
lemma DefinablyComplete.glb :
  ∀ (s : Set M), s.Nonempty → Set.univ.Definable₁ L s
  → BddBelow s → ∃ a, IsGLB s a := by
intro s ⟨x, hx⟩ h2 h3
obtain ⟨r, hr⟩ := DefinablyComplete.lub (L:=L)
(lowerBounds s) h3 (by definability) ⟨x, fun _ h ↦ h hx⟩
exact ⟨r, isLUB_lowerBounds.mp hr⟩
```

2.2 Automating definability proofs

In this section we will discuss our additions on top of mathlib's existing definability library. This includes an `aesop` tactic for solving definability goals.

In the file `Topology.lean` we proved this lemma about the closure of a set in the order topology:

```
lemma closure_order (n : ℕ) : ∀ (A : Set (Fin n → X)), closure A =
  {y | ∀ (a b : (Fin n → X)),
    (∀ (i : Fin n), (a i < y i) ∧ (y i < b i))
    → ∃ x ∈ A, ∀ (i : Fin n), (a i < x i) ∧ (x i < b i)} := by
```

Here the right-hand side is the set of all points $y \in X^n$ for which all boxes containing y intersect A . If A is definable then this set is clearly also first-order definable.

Recall the definition of first-order definability in mathlib:

```
def Definable (s : Set (α → M)) : Prop :=
  ∃ φ : L[[A]].Formula α, s = {v | φ.Realize v}
```

Here φ is a term of the type `L[[A]].Formula α`, which is the type of first-order formulas with constants from A and free variables indexed by α .

Note that the formula in the set-builder notation in `closure_order` is not such a first-order formula, it is a function which sends elements from X^n to the type `Prop`.

To prove definability of sets like in `closure_order` we will need lemmas that turn definability goals into simpler subgoals. This for example includes turning definability of $\{x \mid \varphi(x) \wedge \psi(x)\}$ into definability subgoals $\{x \mid \varphi(x)\}$ and $\{x \mid \psi(x)\}$. Another example would be turning a definability goal $\{x \in$

$M^n \mid \exists c \in M : \varphi(x, c)$ into $\{(x, c) \in M^{n+1} \mid \varphi(x, c)\}$. Here are some of these lemmas that are easy consequences of mathlib's existing definability lemmas:

```

lemma def_conjunction {α : Type*} {P Q : (α → M) → Prop}
  (hp : A.Definable L {a : α → M | P a})
  (hq : A.Definable L {a : α → M | Q a}) :
  A.Definable L {a : α → M | P a ∧ Q a} := hp.inter hq

lemma def_negation {α : Type*} {P : (α → M) → Prop}
  (h : A.Definable L {a : α → M | P a}) :
  A.Definable L {a : α → M | ¬ P a} := h.compl

lemma def_exists {α : Type*} {P : (α → M) → M → Prop}
  (hp : A.Definable L
    {w : α ⊕ Fin 1 → M | P (w ∘ Sum.inl) (w (Sum.inr 0))}) :
  A.Definable L {a : α → M | ∃ x : M, P a x} := ...

lemma def_exists_finite {α β : Type*} [Finite β]
  {P : (α → M) → (β → M) → Prop}
  (h : A.Definable L
    {w : α ⊕ β → M | P (w ∘ Sum.inl) (w ∘ Sum.inr)}) :
  A.Definable L {x : α → M | ∃ a : β → M, P x a} :=
  h.exists_of_finite

```

Using these kinds of lemmas, which are in our file `Definability.lean`, we can prove the definability of the closure:

```

example {n : ℕ} (A : Set (Fin n → X))
  (hA : Set.univ.Definable L A) :
  Set.univ.Definable L (closure A) := by
  rw [closure_order n A]
  simp only [imp_iff_not_or]
  apply Set.def_forall_finite
  apply Set.def_forall_finite
  apply Set.def_disjunction
  · apply Set.def_negation
    apply Set.def_forall_finite_index
    intro i
    apply Set.def_conjunction
    · apply TameGeometry.setOf_lt
      · apply Set.DefinableFun.proj
      · apply Set.DefinableFun.proj
    · apply TameGeometry.setOf_lt
      · apply Set.DefinableFun.proj
      · apply Set.DefinableFun.proj
  · apply Set.def_exists_finite
    apply Set.def_conjunction

```

```

· apply Set.def_preimage_comp
  simp_all only
· apply Set.def_forall_finite_index
  intro i
  apply Set.def_conjunction
· apply TameGeometry.setOf_lt
  · apply Set.DefinableFun.proj
  · apply Set.DefinableFun.proj
· apply TameGeometry.setOf_lt
  · apply Set.DefinableFun.proj
  · apply Set.DefinableFun.proj

```

For the o-minimality and definable completeness theorems we will have to show definability for a lot of sets that are even more complicated. So proving definability like this is not sustainable.

But note that there is no mathematical content in the above proof except `closure_order` at the beginning. Since the formula in the set-builder notation in `closure_order` is a combination of definable operations like $\wedge$, $\vee$, $\rightarrow$ or finite quantification, we can finish the proof by repeatedly applying the corresponding lemmas.

Suppose P is a function from M^n to the type `Prop` such that P is a combination of operations that we know are first-order definable, then there is a trivial proof for the definability of $\{x \mid P x\}$. To automate such proofs we have created a definability tactic using *Aesop* [18], which itself is a tactic for general proof search. *Aesop* stands for “Automated Extensible Search for Obvious Proofs”.

Aesop lets us register different rules which are then used for a tree-based search. Rules are arbitrary Lean tactics that turn a goal into finitely many subgoals. Our rule set consists mostly of applications of lemmas. Rules can be tagged as safe or unsafe. The application of an unsafe rule is allowed to be backtracked, whereas *Aesop* will not backtrack to goals after a safe rule was applied to them.

Safe rules should preserve provability, so if a goal is provable then the subgoals after applying a safe rule should be also provable. Note that what is provable here depends on the rule set we are using.

In the file `Tactic.lean` we register the rule set `Definability` and add the macro `definability` for the *aesop* tactic that uses this rule set:

```

declare_aesop_rule_sets [Definability]

macro "definability" : tactic =>
  '(tactic| aesop (config :=
    { terminal := true, useDefaultSimpSet := false})
    (rule_sets := [$(Lean.mkIdent 'Definability):ident]))

```

Before safe and unsafe rules are applied there is a normalization phase. A normalization rule should simplify the goal and create at most one subgoal.

We use the normalization phase for unfolding definitions or simplifications with mathlib lemmas like `imp_iff_not_or`:

```

attribute [aesop norm unfold (rule_sets := [Definability])]
  Set.DefinableMap
  Set.Definable₁

attribute [aesop norm simp (rule_sets := [Definability])]
  imp_iff_not_or

```

```

theorem imp_iff_not_or : a → b ↔ ¬a ∨ b := ...

```

We can now proof definability of the closure in only two lines with our tactic:

```

example {n : ℕ} (A : Set (Fin n → X))
  (hA : Set.univ.Definable L A) :
  Set.univ.Definable L (closure A) := by
  rw [closure_order n A]
  definability

```

3 Definable completeness

In this chapter $\mathcal{R} = (R, <, \dots)$ is assumed to be a definably complete structure.

3.1 Intermediate value theorem

Theorem 3.1 (Intermediate value theorem). *Let $f: R \rightarrow R$ be definable and continuous on the nonempty interval $[a, b]$. Then for all $c \in (f(a), f(b)) \cup (f(b), f(a))$ there is a $d \in (a, b)$ such that $f(d) = c$.*

Proof. Let $c \in (f(a), f(b))$. The case $c \in (f(b), f(a))$ can be shown similarly. We define

$$S := \{x \mid c \leq f(x)\}$$

and

$$Z := [a, b] \cap S$$

Since f is continuous on $[a, b]$, the set Z is closed.

By definable completeness of R , we get that Z has an infimum $d \in R$. And since Z is closed, d is a minimum of Z .

We now show that $f(d) = c$. From $d \in S$ we have $f(d) \geq c$ and $a < d$.

Assume towards contradiction that $f(d) > c$. By continuity there are $v, w \in R$ with $d \in (v, w)$ such that $c < f(x)$ for all $x \in (v, w) \cap [a, b]$. Since $a < d$, there is $x \in R$ with $\max(v, a) < x < d$. Then $x \in Z$ but also $x < d = \min Z$. This is a contradiction and hence $f(d) = c$.

□

3.2 Extreme value theorem

From now on we will abbreviate “Closed, bounded and definable” with *CBD*.

Remark. A **box** in R^n is the Cartesian product of n intervals (a_i, b_i) where $a_i, b_i \in R$ and $a_i < b_i$. We will be working with the topology on R^n generated by these boxes.

Lemma 3.2. *Let $X \subseteq R^n$ be a definable set. Then the closure $\overline{X}$ is definable.*

Proof. Notice that

$$\overline{X} = \{y \in R^n \mid \text{all boxes containing } y \text{ intersect } X\}$$

It is easy to check that the right-hand side is definable. □

Lemma 3.3. *Let $X \subseteq R^{n+m}$ be CBD and let $\pi: R^{n+m} \rightarrow R^n$ be the projection to the first n coordinates. Then $\pi(X)$ is closed.*

Proof. It suffices to show the case $m = 1$.

Let $y \in \overline{\pi(X)}$. We need to show that $y \in \pi(X)$.

For $A \subseteq R^n$ we define

$$S(A) := \{t \in R \mid \exists z \in A, (z, t) \in X\}$$

Let $B \subseteq R^n$ be a box that contains y . Since y is in $\overline{\pi(X)}$, the box B must intersect $\pi(X)$. It follows that $S(B)$ is nonempty. Boundedness of X implies that $S(B)$ is also bounded. So by definable completeness $S(B)$ has a supremum, which lets us define

$$Z := \{t \in R \mid \text{there is a box } B \text{ with } y \in B \text{ and } t = \sup S(B)\}$$

The set Z is definable, nonempty and bounded below. Hence $c := \inf Z$ exists.

Now we will show that $(y, c) \in X$. Since X is closed, it is enough to show that every box which contains (y, c) intersects X .

Let $B \subseteq R^n$ be a box that contains y and let $a, b \in R$ with $a < c < b$. The product $B \times (a, b)$ is a box containing (y, c) .

Since $c = \inf Z$ and $c < b$, there is a box B_1 that contains y and $c \leq \sup S(B_1) < b$. Let $B_2 := B \cap B_1$, note that this is again a box containing y . From $B_2 \subseteq B_1$ we get $c \leq \sup S(B_2) \leq \sup S(B_1)$. And from $a < c \leq \sup S(B_2)$ we get $t \in S(B_2)$ with $a < t < b$. Since $t \in S(B_2)$, there is $z \in B_2$ with $(z, t) \in X$. We have $B_2 \subseteq B$, hence (z, t) is inside the box $B \times (a, b)$.

□

Definition 3.4. Let $X \subseteq R^n$ be a definable set. A family $(Y_x)_{x \in X}$ of subsets of R^m is called a **definable family** if $\{(x, y) \in R^{n+m} \mid y \in Y_x\}$ is definable.

Definition 3.5. A family $(Y_x)_{x \in X}$ is called **increasing** if

$$r < s \implies Y_r \subseteq Y_s$$

and **decreasing** if

$$r < s \implies Y_r \supseteq Y_s$$

Note that we are using lexicographic ordering here. We say a family is **monotone** if it is increasing or decreasing.

Theorem 3.6 (Monotone Intersection). *Let $(Y_x)_{x \in R}$ be a monotone definable family of subsets of R^n such that each Y_x is nonempty and CBD. Then*

$$\bigcap_{x \in R} Y_x \neq \emptyset$$

Proof. Assume that $(Y_x)_{x \in R}$ is decreasing. The case that $(Y_x)_{x \in R}$ is increasing can be shown similarly. We will do an induction on n .

Assume that $n = 1$. By definable completeness and because each Y_x is nonempty and CBD, we can define

$$g(x) := \min Y_x$$

Since $(Y_x)_{x \in R}$ is decreasing, the function g is increasing:

$$a < b \implies g(a) \leq g(b) \tag{1}$$

The image $g(R)$ is bounded above: pick some $x \in R$ and let $u \in R$ be an upper bound of Y_x . Let $y \in R$. If $x \leq y$ then we have $Y_y \subseteq Y_x$. So u is also an upper bound of Y_y and thus $g(y) \leq u$. If $y < x$, then by (1) we have $g(y) \leq g(x) \leq u$.

Since $g(R)$ is also definable, $g(R)$ has a supremum $c \in R$. Let $x \in R$. We will now show that $c \in Y_x$.

For $y \in (-\infty, x)$ we have $g(y) \leq g(x)$ by (1) and thus c is also a supremum of $g([x, \infty))$.

Notice that $g([x, \infty)) \subseteq Y_x$. So c is the supremum of the nonempty set $g([x, \infty))$, which lies in the closed set Y_x . Hence we get $c \in Y_x$.

For the induction step we will prove the statement for $n + 1$, assuming that it is true for $n \in \mathbb{N}$. By Lemma 3.3, the projection to the first coordinate $\pi_0(Y_x)$ is closed for all $x \in R$. Hence we can apply the base case $n = 1$ to the family $(\pi_0(Y_x))_{x \in R}$ and obtain $a \in \bigcap_{x \in R} \pi_0(Y_x)$. For $x \in R$ we define

$$Z_x := \{y \in R^n \mid (a, y) \in Y_x\}$$

The definable family $(Z_x)_{x \in R}$ is decreasing and each Z_x is nonempty and CBD. So we can apply the induction hypothesis to get $b \in \bigcap_{x \in R} Z_x$ and then $(a, b) \in \bigcap_{x \in R} Y_x$.

□

Theorem 3.7. *Let $f : R^n \rightarrow R^m$ be a definable function. Let $X \subseteq R^n$ be CBD and let f be continuous on X . Then $f(X)$ is also CBD.*

Proof. Definability of $f(X)$ is clear. It suffices to show boundedness for the case $m = 1$. For $x \in R$ we define

$$Y_x = X \cap f^{-1}([x, \infty))$$

Assume towards contradiction that $f(X)$ is not bounded above. Then all Y_x are nonempty and CBD. Furthermore $(Y_x)_{x \in R}$ is a decreasing definable

family. By Theorem 3.6 there is some $r \in \bigcap_{x \in R} Y_x$. For all $x \in R$ we have $x \leq f(r)$. But this contradicts that R has no endpoints. We can similarly show that $f(X)$ is bounded below.

Now we will use Lemma 3.3 to show that $f(X)$ is closed. For that we have to show that the graph of $f|_X$ is CBD. Definability is easy. We already know that $f(X)$ and X are bounded, thus the graph $f|_X$ is bounded as well. Closedness of $f|_X$ follows from continuity and because R is Hausdorff. $\square$

Corollary 3.8 (Extreme Value Theorem). *Let $f : R^n \rightarrow R$ be a definable function. Let X be a nonempty CBD subset of R^n and let f be continuous on X . Then $f(X)$ has a minimum and maximum.*

Proof. By Theorem 3.7 the set $f(X)$ is CBD. So by definable completeness $f(X)$ has an infimum $l \in R$. Since $f(X)$ is closed, l is also a minimum of $f(X)$. The maximum is obtained the same way. $\square$

3.3 Formalization

```

theorem intermediate_value_uIoo {a b : M} (hab : a < b)
  {f : M → M} (h1f : Set.univ.DefinableFun1 L f) (h2f :
    ContinuousOn f (Set.Icc a b)) :
  ∀ u ∈ Set.uIoo (f a) (f b), ∃ c ∈ Set.Ioo a b, f c = u := ...

```

For the formalization of the extreme value theorem we are using the following variable block.

```

variable {M : Type*} [LinearOrder M] [TopologicalSpace M]
[OrderTopology M] [NoMinOrder M] [NoMaxOrder M]
{L : Language} [L.IsOrdered] [L.Structure M]
[M ⊨ L.dlo] [L.OrderedStructure M] [DefinablyComplete L M]

```

Here are the box constructions from Lemma 3.3:

```

private def memBox {n : ℕ} (p : Fin n ⊕ Fin n → M)
  (x : Fin n → M) : Prop :=
  ∀ i, p (Sum.inl i) < x i ∧ x i < p (Sum.inr i)

private def lastFromBox {n : ℕ} (S : Set (Fin (n + 1) → M))
  (p : Fin n ⊕ Fin n → M) : Set M :=
  {m | ∃ y : Fin n → M, memBox p y ∧ Fin.snoc y m ∈ S}

private def boxLUB {n : ℕ} (S : Set (Fin (n + 1) → M))
  (x : Fin n → M) : Set M :=
  {m | ∃ p : Fin n ⊕ Fin n → M, memBox p x ∧
    IsLUB (lastFromBox S p) m}

```

A term $(p : \text{Fin } n \oplus \text{Fin } n \rightarrow M)$ is interpreted as the box defined by the two corners $p \circ \text{Sum.inl}$ and $p \circ \text{Sum.inr}$ from M^n .

$\text{memBox } p \ x$ is true if x is in the box defined by p . The sets lastFromBox and boxLUB are $S(A)$ and Z from Lemma 3.3. $\text{Fin.snoc } y \ m$ is the $n + 1$ tuple where m gets added to the end of the n tuple y .

These three definitions are used to prove Lemma 3.3.

```

lemma cbd_projection_closed (n : ℕ) {m : ℕ} (hn : n ≤ m)
  {s : Set (Fin m → M)} (h1 : IsClosed s) (h2 : BddBelow s)
  (h3 : BddAbove s) (h4 : Set.univ.Definable L s) :
  IsClosed (Fin.take n hn '' s) := ...

```

We proved Theorem 3.6 for increasing and decreasing families. Suppose

$(Y_x)_{x \in M}$ is a definable family and $T \subseteq M^2$ is an irreflexive relation such that $x T y$ or $y T x$ for $x \neq y$ and $Y_x \subseteq Y_y$ for $x T y$. One can check that the proof of Theorem 3.6 still works for this kind of family. In order to avoid having the exact same proof twice we are formalizing only this generalized version of Theorem 3.6.

```
private theorem monotone_inter_1_rel {R : M → M → Prop}
  (h1r : ∀ x, ¬ R x x) (h2r : ∀ {x y}, x ≠ y → R x y ∨ R y x)
  {f : M → Set M} (h1 : def_family_univ_1 L f)
  (h2 : ∀ {x y}, R x y → f x ⊆ f y) (h3 : ∀ x, (f x).Nonempty)
  (h4 : ∀ x, IsClosed (f x))
  (h5 : ∀ x, BddBelow (f x)) (h6 : ∀ x, BddAbove (f x)) :
  (⋂ x, f x).Nonempty := ...
```

With this $n = 1$ version we prove `monotone_inter_rel` for arbitrary $n \in \mathbb{N}$. Using `monotone_inter_rel` and `cbd_projection_closed` we then formalize Theorem 3.7. These lemmas are called `cbd_image_bdd_below`, `cbd_image_bdd_above` and `cbd_image_closed`. Finally, we formalize the extreme value theorem with this.

```
theorem extreme_value {n : ℕ} {s : Set (Fin n → M)}
  (h1 : IsClosed s) (h2 : BddBelow s) (h3 : BddAbove s)
  (h4 : Set.univ.Definable L s) (h5 : s.Nonempty)
  {f : (Fin n → M) → M}
  (h1f : Set.univ.DefinableFun L f) (h2f : ContinuousOn f s) :
  ∃ a ∈ s, ∃ b ∈ s, IsLeast (f '' s) (f a) ∧
  IsGreatest (f '' s) (f b) := ...
```

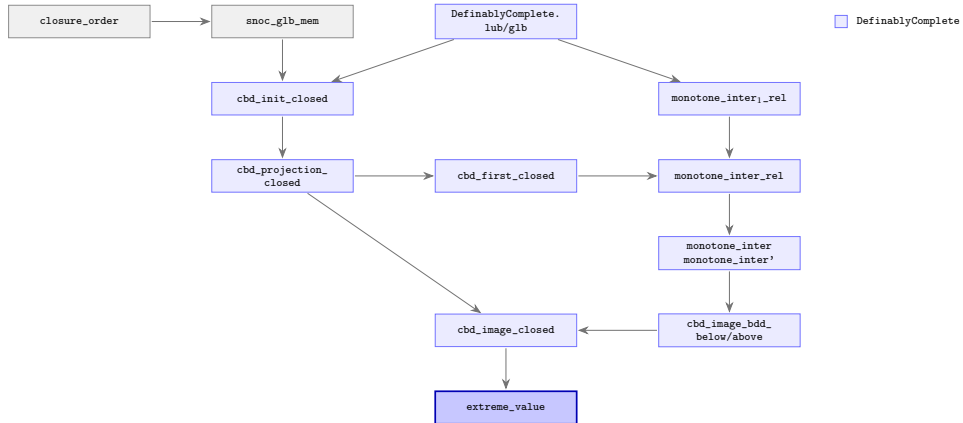


Figure 1: Dependency graph of the Extreme Value Theorem.

4 Monotonicity theorem

In this chapter $\mathcal{R} = (R, <, \dots)$ is assumed to be o-minimal.

4.1 Definable Ramsey theorem

Lemma 4.1. *Let $a, b \in R$ with $a < b$ and let $Y_1, \dots, Y_n \subseteq R$ be definable with $(a, b) \subseteq \bigcup_i Y_i$. Then for all $c \in (a, b)$ there is some $d \in (c, b)$ and $i \in \{1, \dots, n\}$ such that $(c, d) \subseteq Y_i$.*

Proof. Let $c \in (a, b)$. Assume that there is $i \in \{1, \dots, n\}$ and $e \in R$ with $c < e$ and $(c, e) \subseteq Y_i$. Then there is $d \in R$ with $c < d < \min(e, b)$ such that $(c, d) \subseteq Y_i$.

Now assume that no Y_i contains such an interval. Then by o-minimality there is for all $i \in \{1, \dots, n\}$ some $d_i > c$ with $(c, d_i) \cap Y_i = \emptyset$. We define

$$e := \min\{b, d_1, \dots, d_n\}$$

There is some $d \in R$ with $c < d < e$. Then for all i we have $d \in (c, d_i)$ and thus $d \notin Y_i$. This contradicts $(a, b) \subseteq \bigcup_i Y_i$. $\square$

Lemma 4.2. *Let $a, b \in R$ with $a < b$ and let $f: R \rightarrow R$ be definable with $f(x) > x$ for all $x \in (a, b)$. Then there is $d \in (a, b)$ and a nonempty interval $(v, w) \subseteq (a, b)$ such that $x < d < f(x)$ for all $x \in (v, w)$.*

Proof. We define

$$B := \{x \in (a, b) \mid \forall t \in (a, x), f(t) < f(x)\}$$

Assume that B is infinite. Then by o-minimality there is a nonempty interval $(r, s) \subseteq B$. Let $c \in (r, s)$. Since $c \in (a, b)$, we have $f(c) > c$. Thus there is $d \in R$ with

$$c < d < \min(s, f(c))$$

Let $x \in (c, d)$. Then $x \in (r, s) \subseteq B$ and $c \in (a, x)$. By the definition of B this gives us $f(c) < f(x)$. And with $d < f(c)$ we get $x < d < f(x)$.

Now assume that B is finite. There is $b' \in (a, b)$ such that $(a, b') \cap B = \emptyset$.

Let $t \in (a, b')$ and define

$$S := \{x \in R \mid x \in (a, t) \wedge f(t) \leq f(x)\}$$

Since $t \notin B$, there is $r \in (a, t)$ with $f(t) \leq f(r)$ and hence $S \neq \emptyset$. Let $x \in S$. By $x \in (a, b')$ we get $x \notin B$ and thus there is $y \in (a, x)$ with $f(t) \leq f(x) \leq f(y)$. So we have $y < x$ and $y \in S$. This means that S does

not have a minimum. And since a nonempty finite set has a minimum, we conclude that S must be infinite.

Since S is also definable, o-minimality gives us a nonempty interval $(v, w) \subseteq S$. By assumption we have $t < f(t)$, hence there is $d \in R$ with

$$t < d < \min(b, f(t))$$

For $x \in (v, w) \subseteq S$ we now have $x < t < d$ and $d < f(t) \leq f(x)$ and hence also $x < d < f(x)$. $\square$

Theorem 4.3 (Definable Ramsey theorem). *Let $a, b \in R$ with $a < b$ and let $X_1, \dots, X_n \subseteq R^2$ be definable such that $(a, b)^2 \subseteq \bigcup_i X_i$. Then there is a nonempty interval $(v, w) \subseteq (a, b)$ and some $i \in \{1, \dots, n\}$ such that*

$$\{(x, y) \in (v, w)^2 \mid x < y\} \subseteq X_i$$

Proof. For $i \in \{1, \dots, n\}$ we define

$$Z_i = \{x \in (a, b) \mid \exists y \in (x, b) : \{x\} \times (x, y) \subseteq X_i\}$$

Let $x \in (a, b)$. By Lemma 4.1 there is $i \in \{1, \dots, n\}$ and $y \in (x, b)$ such that $\{x\} \times (x, y) \subseteq X_i$. Thus we have $(a, b) = \bigcup_i Z_i$.

Then by Lemma 4.1 there is $i \in \{1, \dots, n\}$ and a nonempty interval $(c, d) \subseteq Z_i$.

For $l \in R$ put

$$S(l) := \{r \in (l, b) \mid \{l\} \times (l, r) \subseteq X_i\} \cup \{l\}$$

and

$$g(l) := \sup S(l)$$

The supremum exists because $S(l)$ is nonempty, bounded and definable. Note that $g(x) > x$ for $x \in (c, d)$. So by Lemma 4.2 there is $m \in (c, d)$ and a nonempty interval $(v, w) \subseteq (c, d)$ such that $x < m < g(x)$ for all $x \in (v, w)$.

Let $x, y \in (v, w)$ with $x < y$. Since $m < g(x)$, there is $r \in S(x)$ with $r > m$. And $r \neq x$ because $x < m$. Thus we have $\{x\} \times (x, r) \subseteq X_i$. And from $y < m < r$ we now get $(x, y) \in X_i$. $\square$

4.2 Proof of the monotonicity theorem

Lemma 4.4. *Let $a, b \in R$ with $a < b$. Let $f: R \rightarrow R$ be definable and strictly monotone on (a, b) . Then there is a nonempty interval $(v, w) \subseteq (a, b)$ on which f is continuous.*

Proof. Assume that f is strictly increasing on (a, b) . The image $f((a, b))$ is a definable infinite set. So by o-minimality it contains some nonempty interval $(c, d) \subseteq f((a, b))$. Let $c', d' \in (c, d)$ with $c' < d'$ and let $v, w \in (a, b)$ with $f(v) = c'$ and $f(w) = d'$. Since f is strictly increasing we have $v < w$. The restriction $f|_{(v, w)}$ is a strictly increasing bijection onto (c', d') . It follows that this restriction is continuous. The case for f strictly decreasing can be shown similarly. $\square$

Notation. Given a function $f: R \rightarrow R$ we will be using the following abbreviations

$$\begin{aligned} A_{=} &:= \{x \in R \mid f \text{ is constant on some interval around } x\} \\ A_{<} &:= \{x \in R \mid f \text{ is strictly increasing on some interval around } x\} \\ A_{>} &:= \{x \in R \mid f \text{ is strictly decreasing on some interval around } x\} \end{aligned}$$

Lemma 4.5. *Let $f: R \rightarrow R$ be definable. Then $B := R \setminus (A_{=} \cup A_{<} \cup A_{>})$ is finite.*

Proof. It is easy to check that B is definable. Assume towards a contradiction that B is infinite. Then there is a nonempty interval $(a, b) \subseteq B$. We define

$$\begin{aligned} X_1 &:= \{(x, y) \in R^2 \mid f(x) = f(y)\} \\ X_2 &:= \{(x, y) \in R^2 \mid f(x) < f(y)\} \\ X_3 &:= \{(x, y) \in R^2 \mid f(x) > f(y)\} \end{aligned}$$

These sets cover $(a, b)^2 \subseteq R^2$. So by Theorem 4.3 there is a nonempty interval $(v, w) \subseteq (a, b)$ and $i \in \{1, 2, 3\}$ such that

$$\{(x, y) \in (v, w)^2 \mid x < y\} \subseteq X_i$$

Assume that $i = 1$. Let $x \in (v, w)$. Now f is constant on the interval (v, w) which contains x and thus $x \in A_{=}$. But this contradicts $x \in (v, w) \subseteq (a, b) \subseteq B$. For $i = 2$ and $i = 3$ we can get a contradiction similarly. $\square$

Lemma 4.6. *Let $f: R \rightarrow R$ be definable. Then there are only finitely many points at which f is discontinuous.*

Proof. Let $D \subseteq R$ be the set of discontinuities of f . This set is definable. Assume towards contradiction that D is infinite. Then there is a nonempty interval $(a, b) \subseteq D$. By Lemma 4.5 the set $B := R \setminus (A_{=} \cup A_{<} \cup A_{>})$ is finite. Hence there is

$$x \in (a, b) \setminus B = (a, b) \cap (A_{=} \cup A_{<} \cup A_{>})$$

So there are $v, w \in R$ with $x \in (v, w)$ such that f is constant or strictly monotone on (v, w) . Then the set $I := (a, b) \cap (v, w) \subseteq D$ is also a nonempty interval on which f is constant or strictly monotone.

If f is constant on I , then it is continuous on I . This contradicts $I \subseteq D$.

If f is strictly monotone on I , then by Lemma 4.4 there is a nonempty interval $(r, s) \subseteq I$ on which f is continuous. This again contradicts $I \subseteq D$. $\square$

Lemma 4.7. *Let $I \subseteq R$ be an interval and let $f: R \rightarrow R$ be definable. Let $T \subseteq R^2$ be a transitive definable relation on R such that for all $x \in I$ there are $a, b \in R$ with $x \in (a, b)$ and*

$$v < w \implies f(v) T f(w) \quad (1)$$

for all $v, w \in (a, b)$. Then (1) also holds for all $v, w \in I$.

Proof. Assume towards a contradiction that there are $v, w \in I$ with $v < w$ and $\neg f(v) T f(w)$. Put

$$Z := \{z \in (v, w] \mid \neg f(v) T f(z)\}$$

This set is nonempty since $w \in Z$. It is bounded below by v . And since it is also definable, Z has an infimum. From $\inf Z \in [v, w]$, we get $\inf Z \in I$.

By assumption there are $a, b \in R$ with $\inf Z \in (a, b)$ and $f(r) T f(s)$ for $r, s \in (a, b)$ with $r < s$. There is $q \in Z$ with $\inf Z \leq q < b$. We will now show that $f(v) T f(q)$, which contradicts $q \in Z$.

If $v = \inf Z$, then $v, q \in (a, b)$ with $v < q$. Hence we get $f(v) T f(q)$.

If $v < \inf Z$, then there is $p \in R$ with $\max(v, a) < p < \inf Z$. We have $p \notin Z$ and $p \in (v, w]$. So we have $f(v) T f(p)$. From $p, q \in (a, b)$ and $p < q$ we get $f(p) T f(q)$. Now transitivity gives us $f(v) T f(q)$. $\square$

Theorem 4.8 (Monotonicity Theorem). *Let $f: R \rightarrow R$ be a definable function. Then there is a finite family $I \subseteq \mathcal{P}(R)$ of open intervals and points from R such that $\bigcup_{S \in I} S = R$ and for each $S \in I$ the function f is continuous on S and it is constant or strictly monotone on S .*

Proof. Let $D \subseteq R$ be the set of discontinuities of f . By Lemma 4.6 the set D is finite.

The sets $A_{=} \setminus D$, $A_{<} \setminus D$ and $A_{>} \setminus D$ are definable and thus by o-minimality each of them is the union of a finite family of intervals and points. We will call these families $I_{=}$, $I_{<}$ and $I_{>}$.

Let $B := R \setminus (A_{=} \cup A_{<} \cup A_{>})$. By Lemma 4.5 the set $X := B \cup D$ is finite. We define

$$I := I_{=} \cup I_{<} \cup I_{>} \cup \{\{p\} \mid p \in X\}$$

This again is a finite family of intervals and points. Note that $\bigcup_{S \in I} S = R$.

Let $S \in I$. If S is a point, then f is trivially continuous and constant on S . So we can assume that S is an interval with $S \in I_- \cup I_< \cup I_>$. We have $S \cap D = \emptyset$ and thus f is continuous on S . Using Lemma 4.7 for one of the relations $<$, $>$, or $=$ we get that f is strictly monotone or constant on S . $\square$

4.3 Formalization

We are working with the following variable block

```
variable {L : Language} {M : Type*} [L.IsOrdered] [L.Structure M]
  [LinearOrder M] [L.OrderedStructure M] [M ⊨ L.dlo]
  [DenselyOrdered M] [OMinimal L M] [NoMinOrder M] [NoMaxOrder M]
  [TopologicalSpace M] [OrderTopology M]
```

We registered an instance of `DefinablyComplete` for o-minimal structures.

```
instance [OMinimal L M] [NoMaxOrder M]
  [NoMinOrder M] [DenselyOrdered M] :
  DefinablyComplete L M where
  lub s h1 h2 h3 := has_lub (M:=M) s (OMinimal.is_omimal h2) h1 h3
```

So whenever we are working with `[OMinimal L M]`, Lean's typeclass inference also provides us `[DefinablyComplete L M]` automatically.

In the proofs o-minimality is mostly used to get a nonempty open interval inside an infinite definable set:

```
lemma isTame_infinite_has_iao [NoMaxOrder M] [NoMinOrder M]
  {s : Set M} (h1s : IsTame s) (h2s : s.Infinite) :
  ∃ a b, a < b ∧ Set.Ioo a b ⊆ s := ...
```

In Theorem 4.3 we used the function $g(l) := \sup S(l)$. We formalized this function's existence and definability as a separate lemma:

```
lemma exists_definableFun_lub {s : M → Set M}
  (h1s : def_family_univ₁ L s)
  (h2s : ∀ x, (s x).Nonempty) (h3s : ∀ x, BddAbove (s x)) :
  ∃ g : M → M, Set.univ.DefinableFun₁ L g ∧
  ∀ x, IsLUB (s x) (g x) := ...
```

This lemma also only uses definable completeness and no o-minimality.

Lemma 4.1 and Lemma 4.2 are formalized in `OMinimal.lean` and they are called `iao_def_covering` and `interior_def_fun`. Using these lemmas we then prove Theorem 4.3.

```

theorem definable_ramsey {n : ℕ} {a b : M} (hab : a < b)
  (X : Fin n → Set (Fin 2 → M))
  (h1X : ∀ i, Set.univ.Definable L (X i))
  (h2X : ∀ x y,
    x ∈ Set.Ioo a b → y ∈ Set.Ioo a b → ∃ i, ![x, y] ∈ X i) :
  ∃ v w, v < w ∧ Set.Ioo v w ⊆ Set.Ioo a b ∧
  ∃ i, ∀ x y, x < y → x ∈ Set.Ioo v w → y ∈ Set.Ioo v w →
  ![x, y] ∈ X i := ...

```

Here the term $![x, y]$ is simply the tuple (x, y) from $\text{Fin } 2 \rightarrow M$.

We define $A_{\text{rel}} R f$ as the set of points for which $a < b \rightarrow R f(a) f(b)$ holds in an interval around them. With this we then define $A_{<}$, $A_{>}$ and $A_{=}$.

```

private abbrev A_rel (R : M → M → Prop) (f : M → M) : Set M :=
  {x : M | ∃ v w, x ∈ Set.Ioo v w ∧ ∀ a ∈ Set.Ioo v w,
    ∀ b ∈ Set.Ioo v w, a < b → R (f a) (f b)}

private abbrev A_eq (f : M → M) : Set M := A_rel (·=·) f

private abbrev A_lt (f : M → M) : Set M := A_rel (·<·) f

private abbrev A_gt (f : M → M) : Set M := A_rel (·>·) f

```

Next Lemma 4.5 and Lemma 4.6 are formalized.

```

private lemma B_finite {f : M → M}
  (h : Set.univ.DefinableFun1 L f) :
  (A_eq f ∪ A_lt f ∪ A_gt f)c.Finite := by

lemma discontinuities_finite {f : M → M}
  (hf : Set.univ.DefinableFun1 L f) :
  {x : M | ¬ContinuousAt f x}.Finite := by

```

Our definability tactic is especially useful for `B_finite`. Definability of $(A_{\text{eq}} f \cup A_{\text{lt}} f \cup A_{\text{gt}} f)^c$ is shown with `(by definability)`.

`discontinuities_finite` uses the lemma

`continuous_of_def_strictMonoOn_or_strictAntiOn`, which is the formalization of Lemma 4.4.

We formalize Lemma 4.7 for order connected sets instead of only for intervals. A set s is called order connected if $[x, y] \subseteq s$ for all $x, y \in s$. For example the set $\mathbb{Q} \cap (0, \sqrt{2}) \subseteq \mathbb{Q}$ is order connected but not an interval.

```
lemma localRel_to_global [DefinablyComplete L M]
  {s : Set M} (hs : s.OrdConnected) {f : M → M}
  (hf : Set.univ.DefinableFun1 L f)
  {r : M → M → Prop} [IsTrans M r]
  (h1r : Set.univ.Definable L {v : Fin 2 → M | r (v 0) (v 1)})
  (h2r : ∀ x ∈ s, ∃ a b, x ∈ Set.Ioo a b ∧ ∀ s ∈ Set.Ioo a b,
    ∀ t ∈ Set.Ioo a b, s < t → (r (f s) (f t))) :
  ∀ x ∈ s, ∀ y ∈ s, x < y → r (f x) (f y) := ...
```

Finally, we formalize the Monotonicity theorem.

```
theorem monotonicity_theorem {f : M → M}
  (hf : Set.univ.DefinableFun1 L f) :
  ∃ A : Finset (Set M), (⋃ s ∈ A, s = Set.univ) ∧ ∀ s ∈ A,
    IsBasic s ∧ ContinuousOn f s ∧ ((f '' s).Subsingleton ∨
    StrictMonoOn f s ∨ StrictAntiOn f s) := ...
```

$(f '' s).Subsingleton$ means that $f(s)$ has at most one element, i.e. it is constant on s .

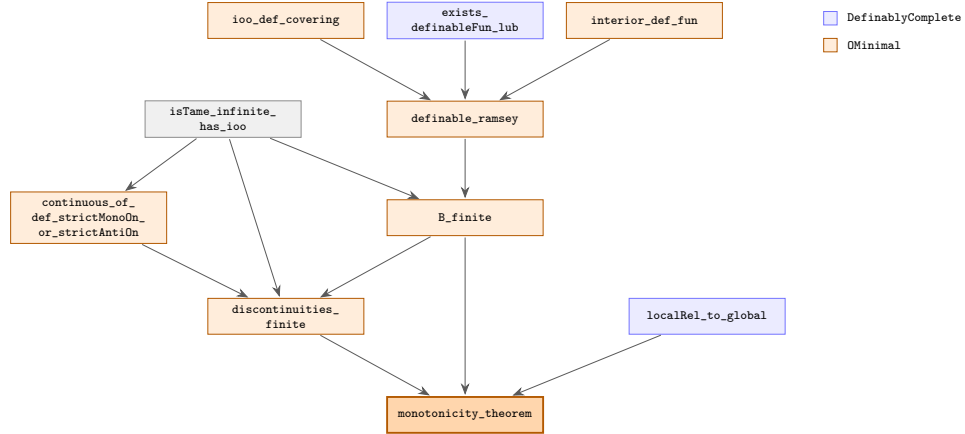


Figure 2: Dependency graph of the Monotonicity Theorem.

Conclusion

We formalized the monotonicity theorem for o-minimal structures and the intermediate value theorem and extreme value theorem for definably complete structures in Lean. We built an Aesop tactic for automating easy definability goals.

In total, the o-minimality results, including the monotonicity theorem, are around 500 lines of code. The definable completeness results are about 550 lines of code and the definability additions are about 350 lines of code.

Future work could formalize the cell decomposition theorem or prove o-minimality for some structure. If at some point quantifier elimination gets added to mathlib, o-minimality of the real field $(\mathbb{R}, <, +, \cdot)$ could be formalized.

Furthermore, a definability tactic like the one we implemented might be a good addition to mathlib.

Bibliography

- [1] Alexandre Grothendieck. “Esquisse d’un programme”. In: *Geometric Galois Actions, 1*. London Mathematical Society Lecture Note Series 242. Written in 1984. Cambridge University Press, 1997, pp. 5–48.
- [2] Anand Pillay and Charles Steinhorn. “Definable Sets in Ordered Structures”. In: *Bulletin of the American Mathematical Society (New Series)* 11.1 (1984), pp. 159–162.
- [3] Lou van den Dries. *Tame Topology and O-minimal Structures*. London Mathematical Society Lecture Note Series 248. Cambridge University Press, 1998.
- [4] Jonathan Pila, Ananth N. Shankar, and Jacob Tsimerman. *Canonical heights on Shimura varieties and the André-Oort conjecture*. appendix by Hélène Esnault and Michael Groechenig. 2021. arXiv: 2109.08788 [math.NT].
- [5] Jonathan Pila and Umberto Zannier. “Rational points in periodic analytic sets and the Manin-Mumford conjecture”. In: *Atti Accad. Naz. Lincei Rend. Lincei Mat. Appl.* 19.2 (2008), pp. 149–162.
- [6] International Mathematical Union. *Fields Medal 2026: Jacob Tsimerman*. https://www.mathunion.org/fileadmin/documents/2026-07/Jacob_Tsimerman_Citations.pdf. 2026.
- [7] A. J. Wilkie. “Model completeness results for expansions of the ordered field of real numbers by restricted Pfaffian functions and the exponential function”. In: *Journal of the American Mathematical Society* 9.4 (1996), pp. 1051–1095. DOI: 10.1090/S0894-0347-96-00216-0.
- [8] Philipp Hieronymi. *Tame Geometry*. Lecture notes. Universität Bonn. 2024.
- [9] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. DOI: 10.1007/978-3-030-79876-5_37. URL: https://doi.org/10.1007/978-3-030-79876-5_37.
- [10] Lars Becker et al. “Formalizing Carleson’s Theorem in Lean”. 2026. URL: <https://florisvandoorn.com/carleson/paper.pdf>.
- [11] Kevin Buzzard and Richard Taylor. *Towards a Lean proof of Fermat’s Last Theorem*. Blueprint of the Lean formalization project. Lean code: <https://github.com/ImperialCollegeLondon/FLT>. 2024. URL: <https://imperialcollegelondon.github.io/FLT/blueprint/index.html>.

- [12] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*. https://lean-lang.org/theorem_proving_in_lean4/.
- [13] William A. Howard. “The Formulae-as-Types Notion of Construction”. In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Academic Press, 1980, pp. 479–490.
- [14] Nicolas D. Goodman and John Myhill. “Choice Implies Excluded Middle”. In: *Zeitschrift für Mathematische Logik und Grundlagen der Mathematik* 24.25–30 (1978), p. 461. DOI: 10.1002/malq.19780242514.
- [15] Mario Carneiro. “The Type Theory of Lean”. MA thesis. Carnegie Mellon University, 2019. URL: <https://github.com/digama0/lean-type-theory>.
- [16] Jesse Michael Han and Floris van Doorn. “A formal proof of the independence of the continuum hypothesis”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, Jan. 2020. DOI: 10.1145/3372885.3373826.
- [17] Reid Barton and Johan Commelin. *lean-omin*. <https://github.com/rwbarton/lean-omin>. 2020.
- [18] Jannis Limperg and Asta Halkjær From. “Aesop: White-Box Best-First Proof Search for Lean”. In: *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, Jan. 2023, pp. 253–266. DOI: 10.1145/3573105.3575671.